

夏のICN研ワークショップ (Cefore/CeforeEmu)

2026/08/27

コンテンツ

■ ICN/Cefore概要

■ Cefore Emulator (CeforeEmu)

- 概要・サンプル実行
- CeforeEmu 実例
 - Reflexive forwarding (Push通信) を試す
 - ワイヤレス環境に対応させる
 - Vision AI/Computer VisionシステムをICNで動かす
 - まずは、Emulationなし(Client ← Cefore ← Publisher)で動かす
 - 次に、Emulationありで動かす (Client ← Cefore ← Cefore ← Cefore ← Publisher)
 - 既存研究方式、自身の方式を組み入れて評価
 - etc...

ICN/Cefore概要

ヘンゼルとグレーテルとは？

- グリム童話に収録されている作品
- 主人公（ヘンゼル）が妹（グレーテル）と森から家に帰るために取った戦略とは？



引用：

http://retoro-enikki.cocolog-nifty.com/blog/2008/06/post_cbd4.html

Wikipediaより：

<https://ja.wikipedia.org/wiki/%E3%83%98%E3%83%B3%E3%82%BC%E3%83%AB%E3%81%A8%E3%82%B0%E3%83%AC%E3%83%BC%E3%83%86%E3%83%AB>

「パンくず」が学術論文に？

ヘンゼルとグレーテルが使った
「パンくず」戦略は、
著名な論文で使われており、
情報指向ネットワーク技術
に活用されている。

なぜ情報指向ネットワークで
「パンくず戦略」
が有効なのか？
どのようにして使うのか？

Breadcrumbs: efficient, best-effort content location in cache networks

Elisha J. Rosensweig
Department of Computer Science
University of Massachusetts
Amherst, Massachusetts 01003-9264
Email: elisha@cs.umass.edu

Jim Kurose
Department of Computer Science
University of Massachusetts
Amherst, Massachusetts 01003-9264
Email: kurose@cs.umass.edu

Abstract—For several years, web caching has been used to meet the ever-increasing Web access loads. A fundamental capability of all such systems is that of inter-cache coordination, which can be divided into two main types: explicit and implicit coordination. While the former allows for greater control over resource allocation, the latter does not suffer from the additional communication overhead needed for coordination.

In this paper, we consider a network in which each router has a local cache that caches files passing through it. By additionally storing minimal information regarding caching history, we develop a simple content caching, location, and routing systems that adopts an implicit, transparent, and best-effort approach towards caching. Though only best effort, the policy outperforms classic policies that allow explicit coordination between caches.

I. INTRODUCTION

For several years, web caching has been used to meet the ever increasing Web access loads [1]. More recently, advocates of content-centric networking [2], [3] have argued for raising the level of abstraction of the atomic unit of data that is stored and forwarded within the network from a packet to a file, or other higher-level content unit. In both cases, content storage, location, and forwarding within the network are of central concern.

Although content storage (caching) systems come in many forms and flavors, one fundamental capability of all such systems is that of coordination, which can be divided into two main types: explicit and implicit. With explicit coordination, caches share their state (or state summaries), and additional information such as access patterns and content popularity [4] with each other. Using this information, each cache determines

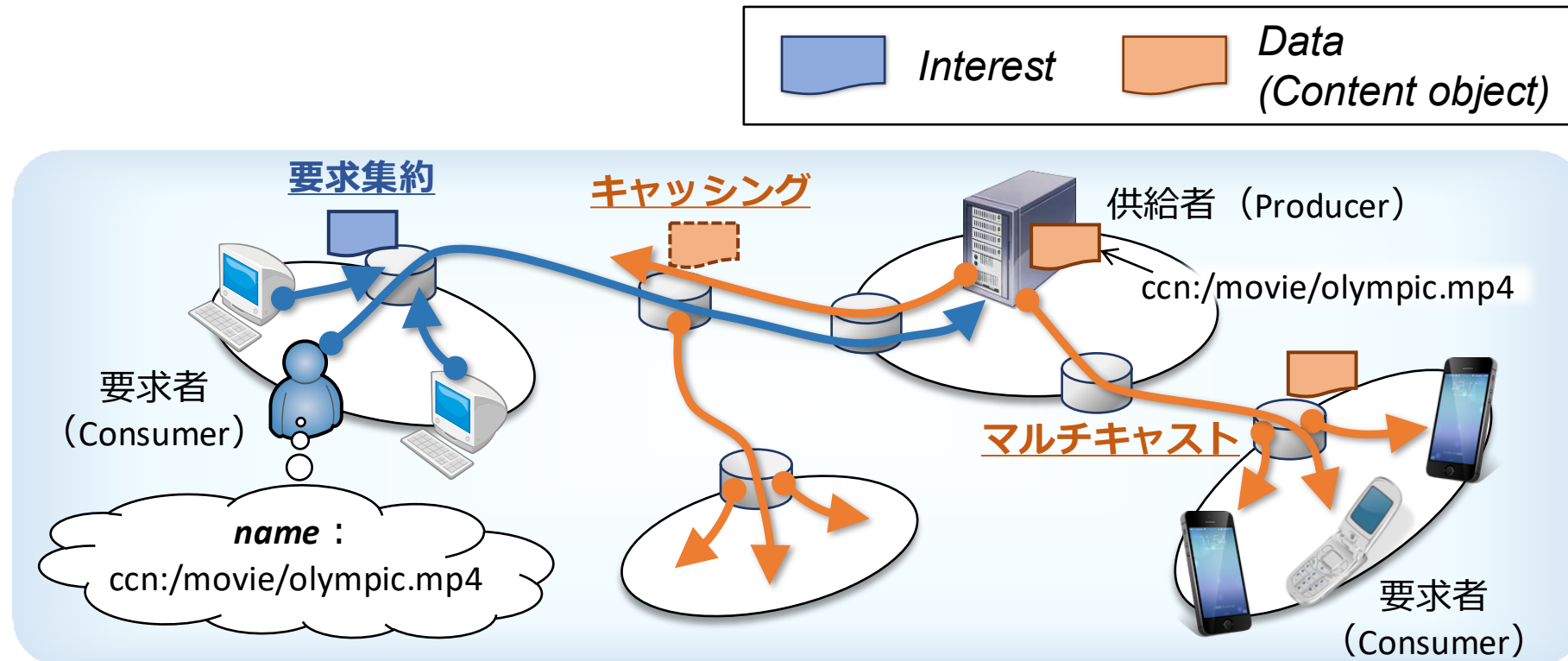
hierarchical cache systems [7], where caches are arranged in a tree-like structure. Requests start out at the leaves of the tree, and are routed towards the root until the content is found, either in the tree or at an external source (via the root) when the content is not present in the tree. The system topology provides for implicit cache coordination to take place, as the position of each cache will cause it to hold different types of files and so manage resources efficiently.

In this paper, we describe a simple content caching, location, and routing system that adopts an implicit, transparent, and best-effort approach towards caching. We consider a network scenario in which each router has a local cache that caches files passing through it. Requests for a file are routed initially towards the source of the file, and en-route check at each router whether a copy of the file is present at its cache, and download it directly from there if found. Such caches are commonly referred to as Transparent En-Route Caches (TERC) - "transparent" in that neither the user nor the server are aware that any such cache exists, and "en-route" since they are accessed during a standard request, on the path to the server [8][9]. The Breadcrumbs approach described in this paper is "best effort" in that coordination is implicit, and forwarded requests may (or may not) locate content while being routed among the caches; if not located, content can always be eventually retrieved from the source. Only a minimal amount of per-file information (termed "breadcrumb") is used in locating content. A breadcrumb stores the most recent direction and time that a file was forwarded in the past, thus tying content routing with content location and caching. We find that although our

IEEE INFOCOM, 2009

情報指向ネットワーク技術 (ICN/CCN)

- **ホスト中心ではなく情報/コンテンツ中心**のネットワークアーキテクチャ
 - IP アドレスではなくコンテンツ名を使用
- **コンテンツを効率的に配布・取得**するための仕組みをネットワークの機能（アプリケーション層の機能ではなく）としてサポート



ICNの概略図

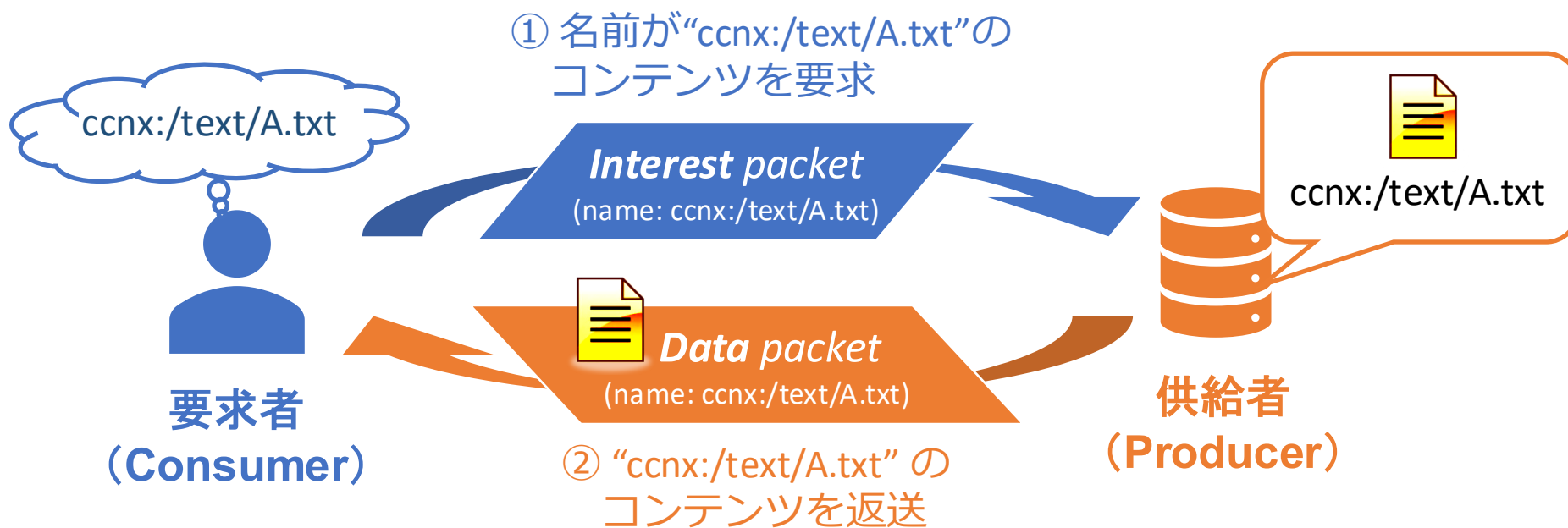
コンテンツ名アドレス

- コンテンツ（のチャンク）ごとに名前を割り当てる
- URL のような階層構成を持つ
 - プレフィックス単位の経路制御が可能
 - バージョン情報やコンテンツのチャンク番号も含める



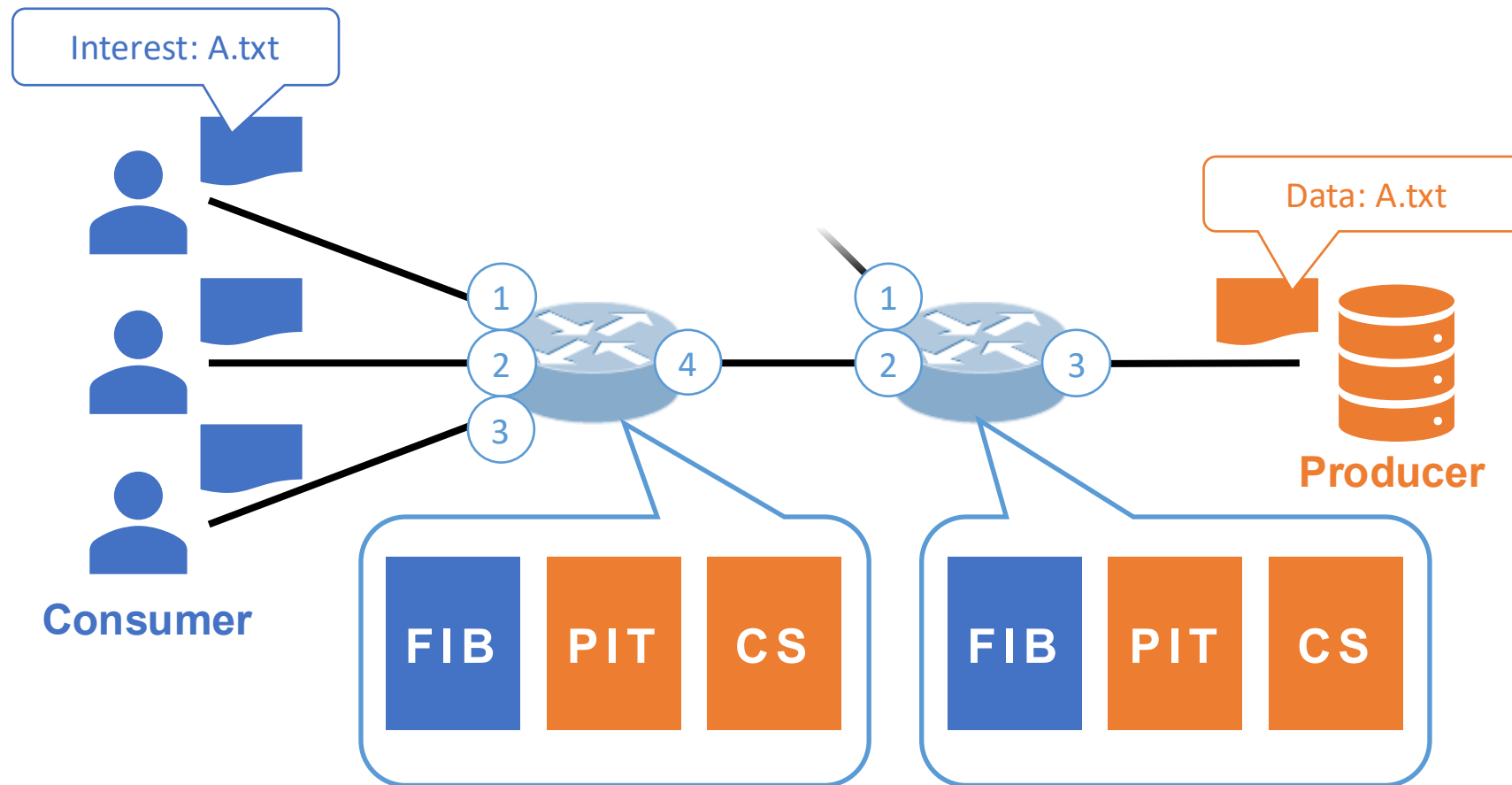
コンテンツ名に基づく通信

- 消費者（Consumer）と供給者（Producer）で通信
 - IP アドレスのような場所（通信相手）の情報は不要
- 2 種類のパケットを用いて通信
 - **Interest:** コンテンツを要求するためのパケット
 - **Data:** コンテンツを返送するためのパケット



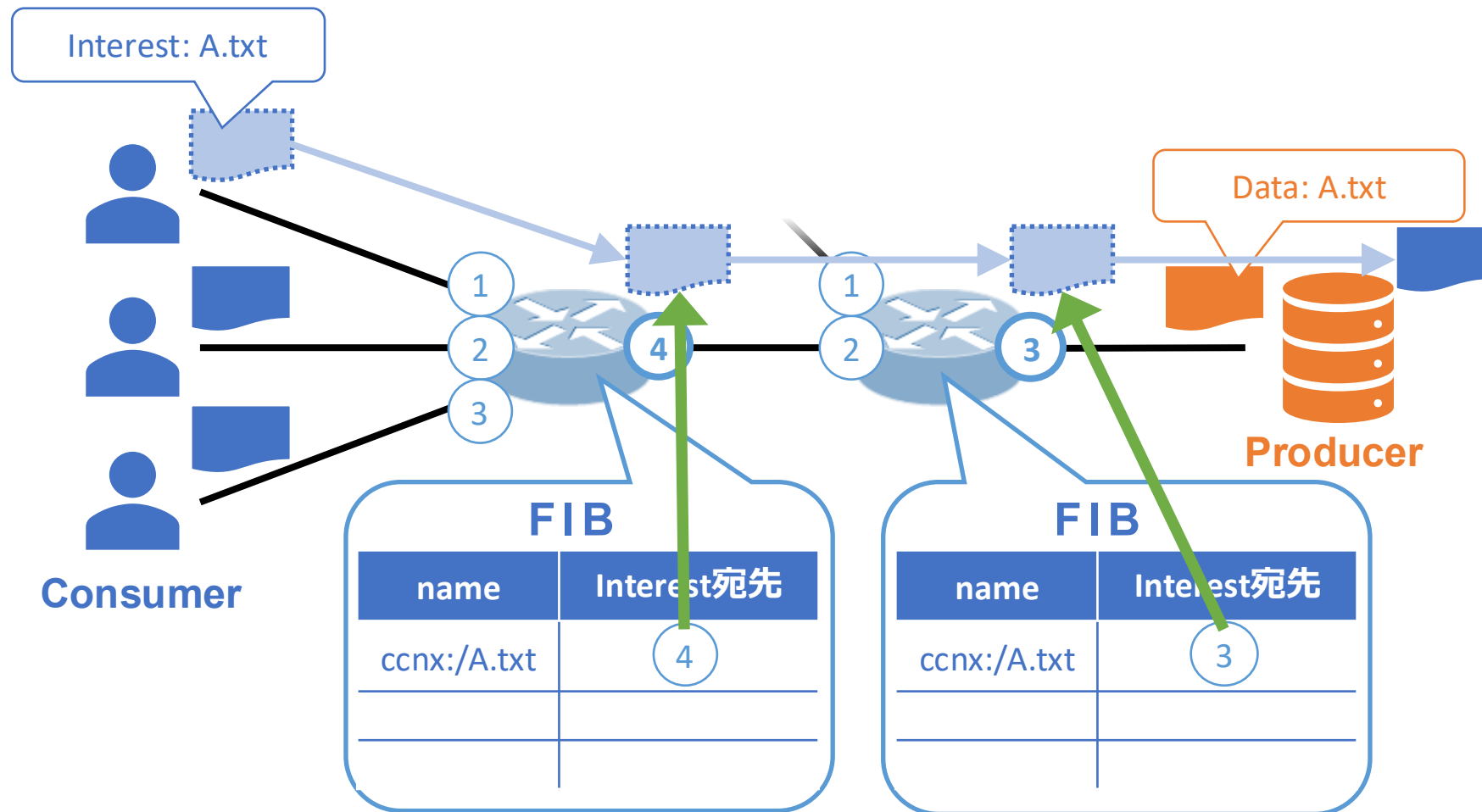
ICN の通信例

- ICN ルータ（中継ノード）はどのように通信を効率化するのか？
→例：3人の Consumer が ccnx:/A.txt を要求



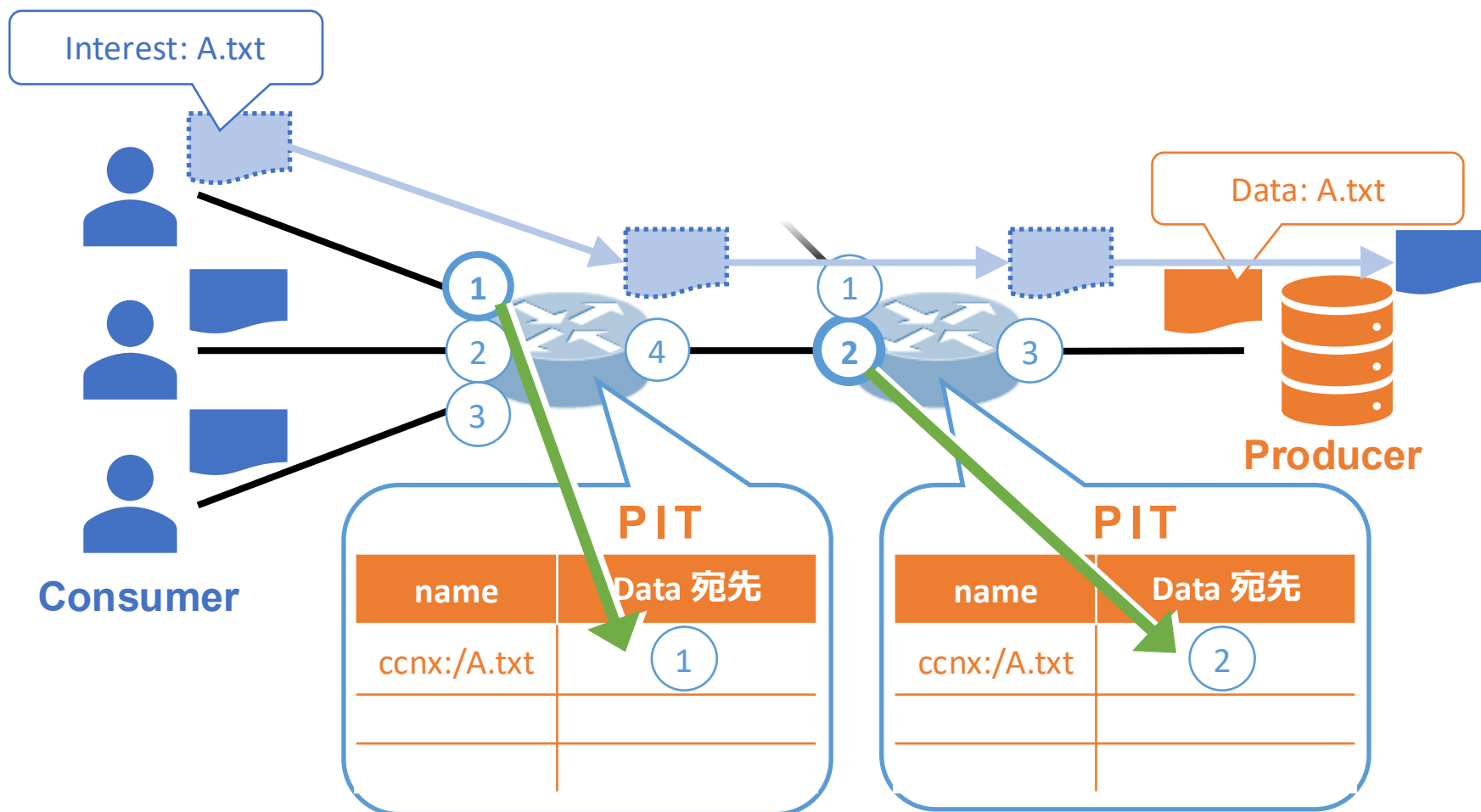
(1) Interest 転送

- 1 人目の Consumer が Interest パケットを送出する
 - ICN ルータは **Forwarding Information Base (FIB)** に従って Interest パケットを転送する



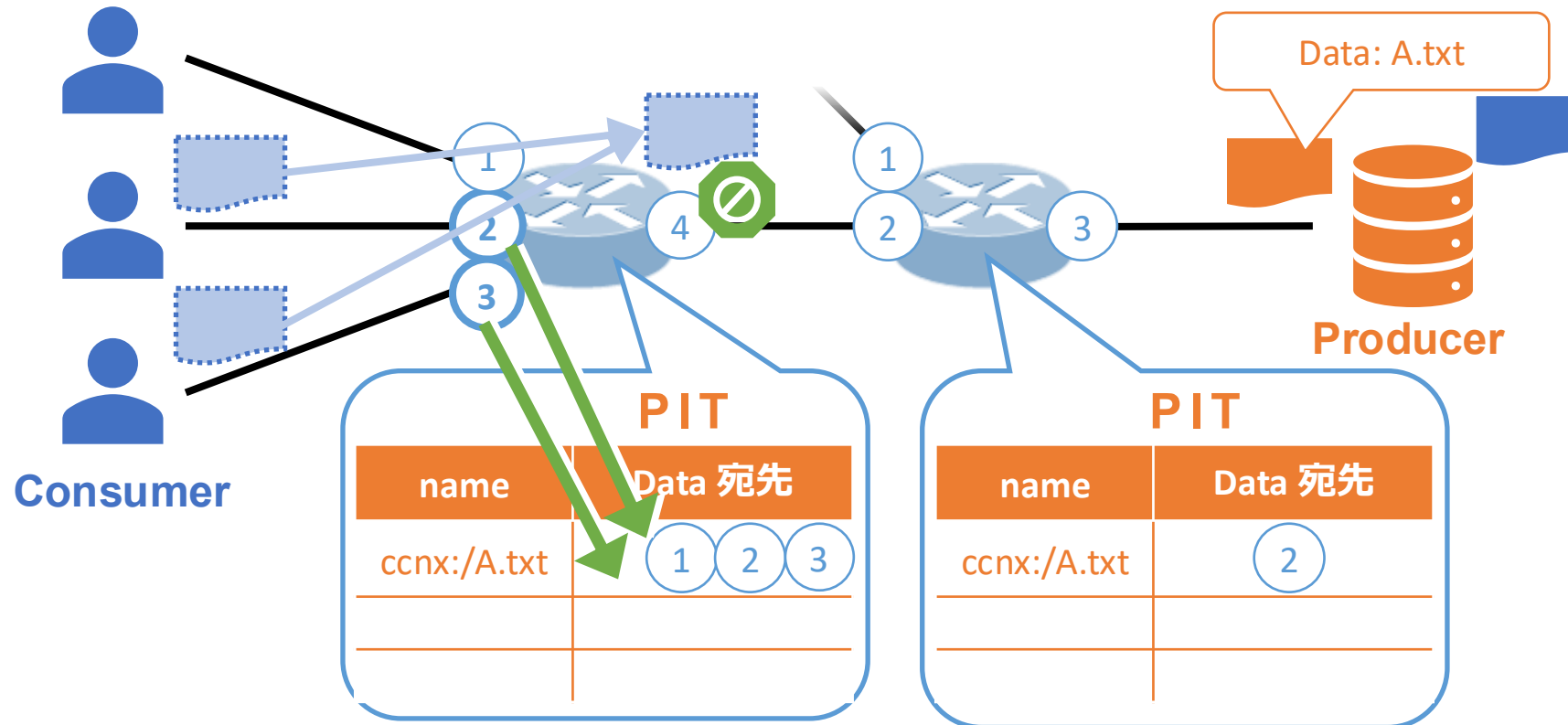
(2) 要求待ち Interest の記憶

- ルータは Interest を転送すると同時に、要求のあったポートを **Pending Interest Table (PIT)** に記憶する
 - 後で Data パケットの返送先として利用する



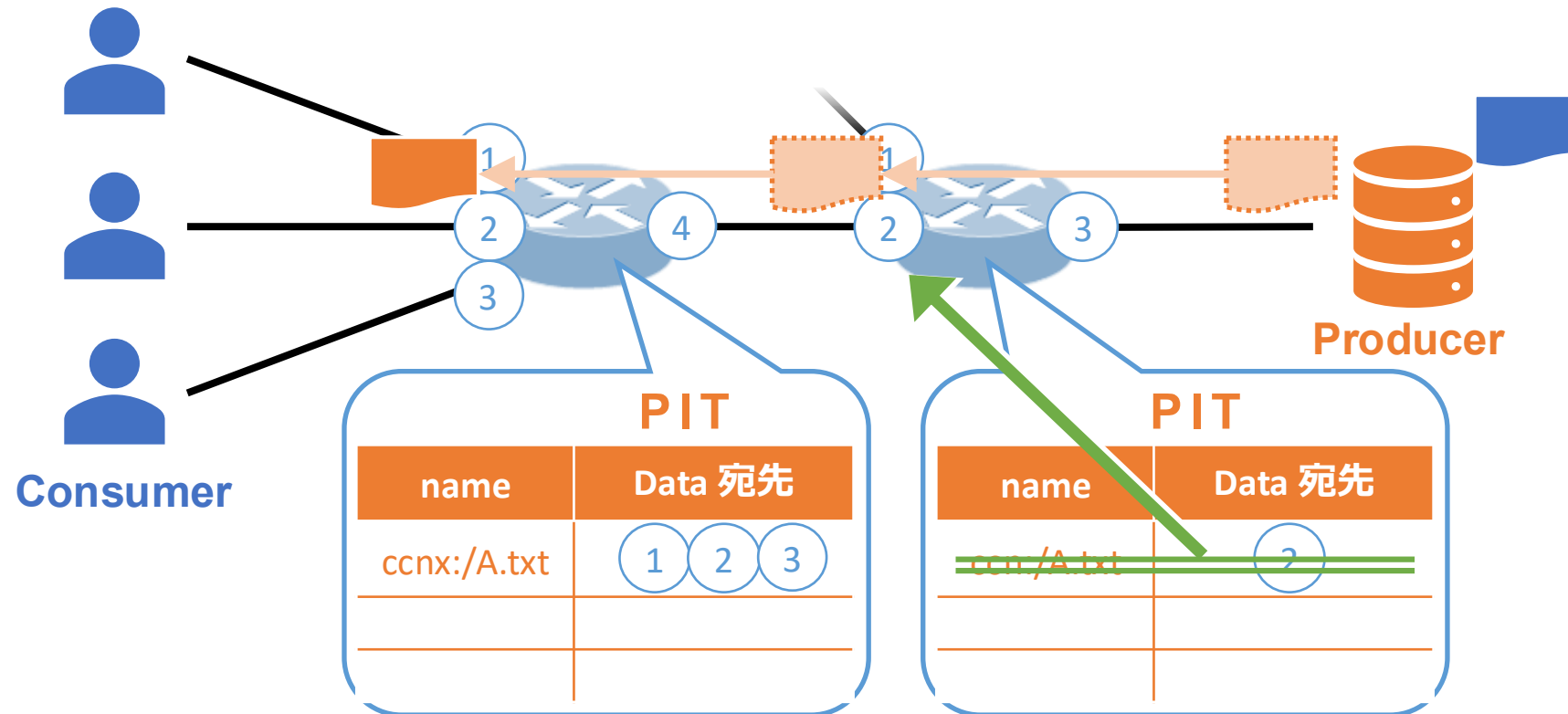
(3) 重複 Interest の集約

- Data パケットが返ってくるまでの間に、他の 2 人が Interest パケットを送出すると、**ルータが集約する(要求集約)**
 - ルータは以前と同じ要求だと分かるので転送はしない
 - PIT に 3 ポート分の情報が記憶される



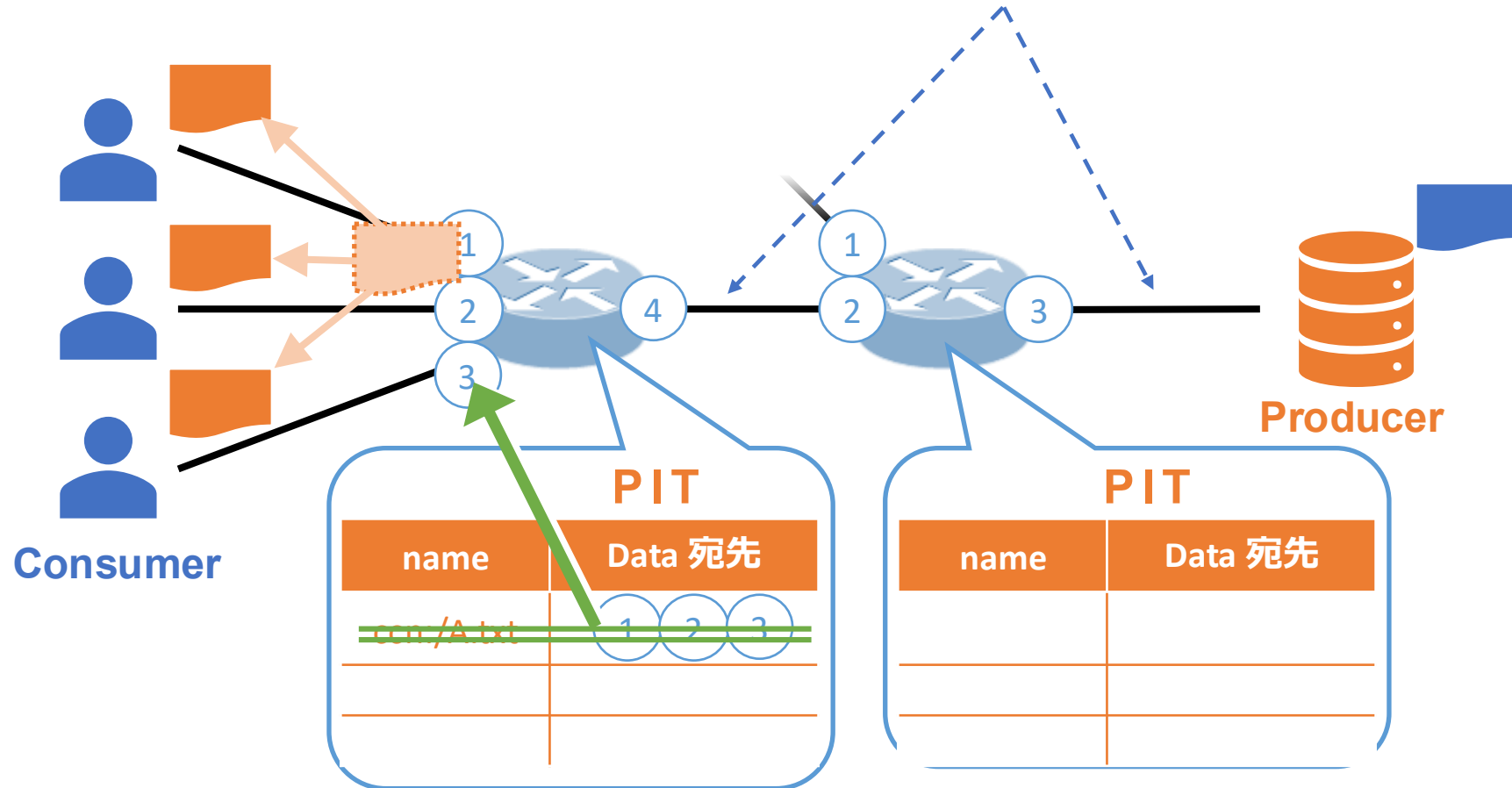
(4) Data パケットの返送

- Producer は Data パケットを返送する
 - Data パケットが転送されると要求が満たされたとみなし、**PIT エントリは削除される**
 - PIT エントリが無い限り Data パケットは転送されない



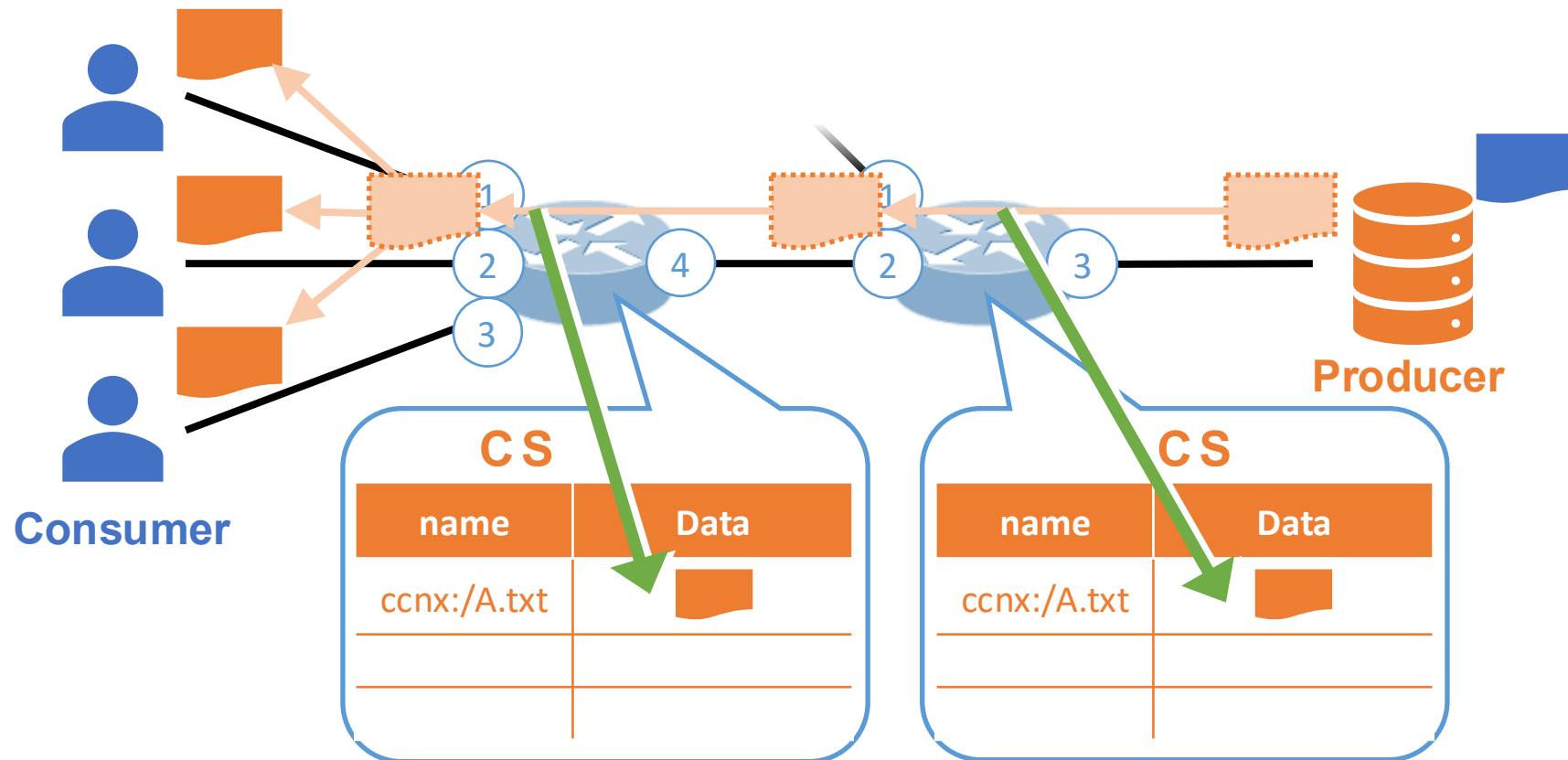
(5) Data パケットのマルチキャスト

- PIT に複数の要求が集約されている場合は、すべての要求に対して**マルチキャストされる**
 - 結果的に Producer が送出するパケットは1つで済み、**サーバの負荷が軽減される（リンクの負荷も軽減される）**



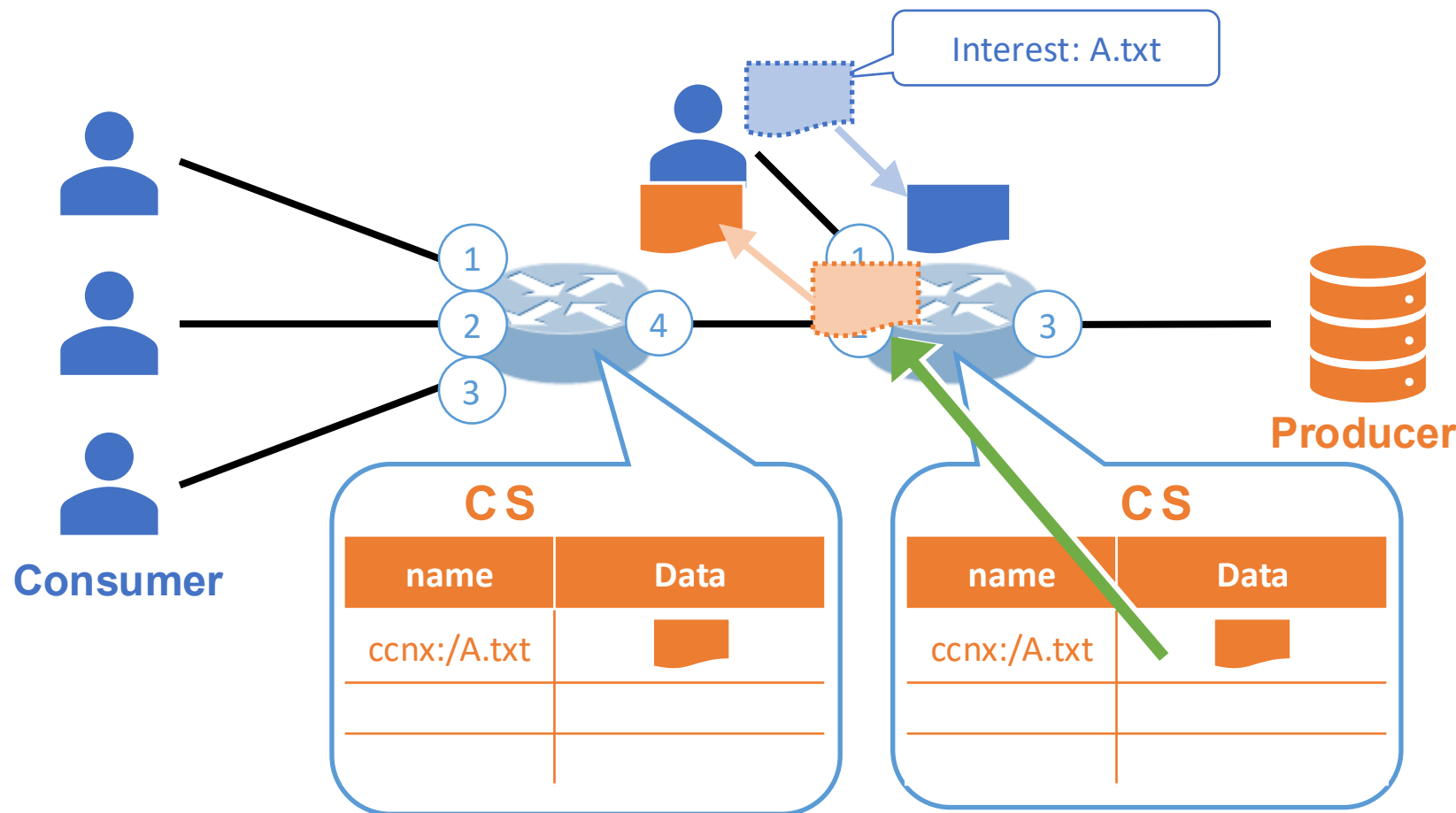
(6) Data パケットのキャッシュ

- Data パケットを転送したルータはその Data パケットを **Content Store (CS)** にキャッシュする



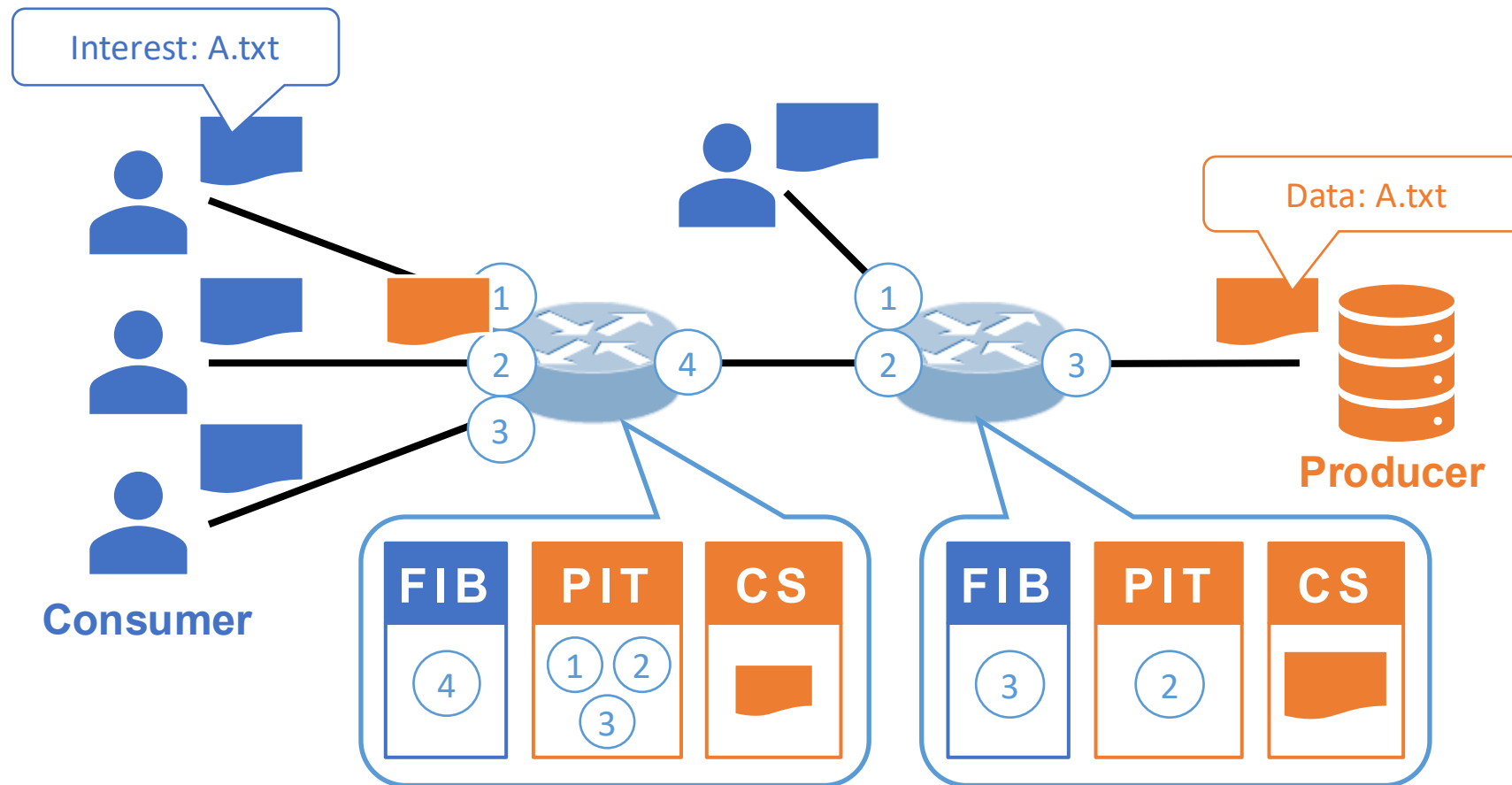
(7) ルータによるキャッシュ応答

- 別の Consumer が後から同じ A.txt を要求
 - ルータは**コンテンツ名を見れば同じだと分かる**ので Producer に転送せず**直接 Data パケットを返送する**



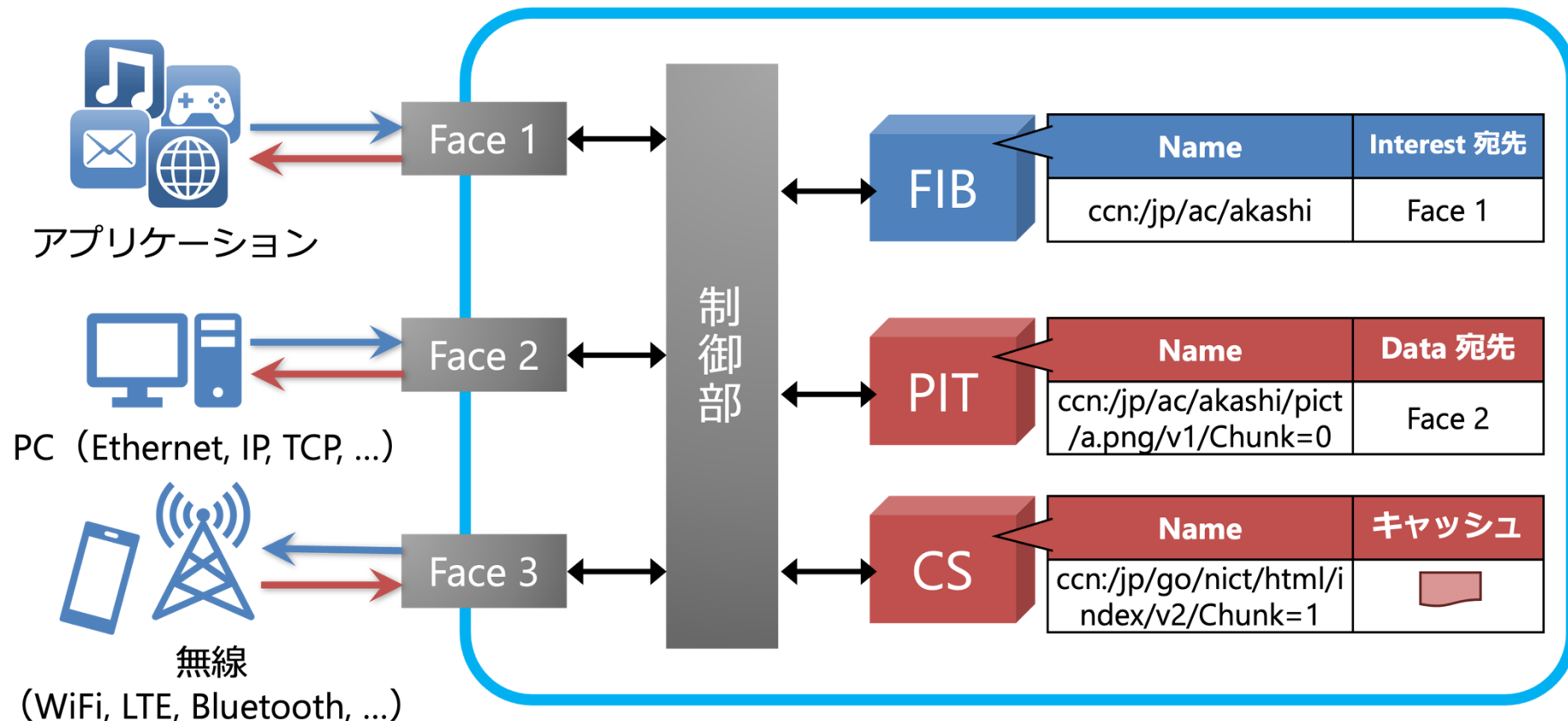
ICN/CCN の通信例：まとめ

- ICN/CCN ルータは FIB・PIT・CS の 3 つのテーブルによって効率的なコンテンツの取得・配布をサポート



ICN ルータモデル

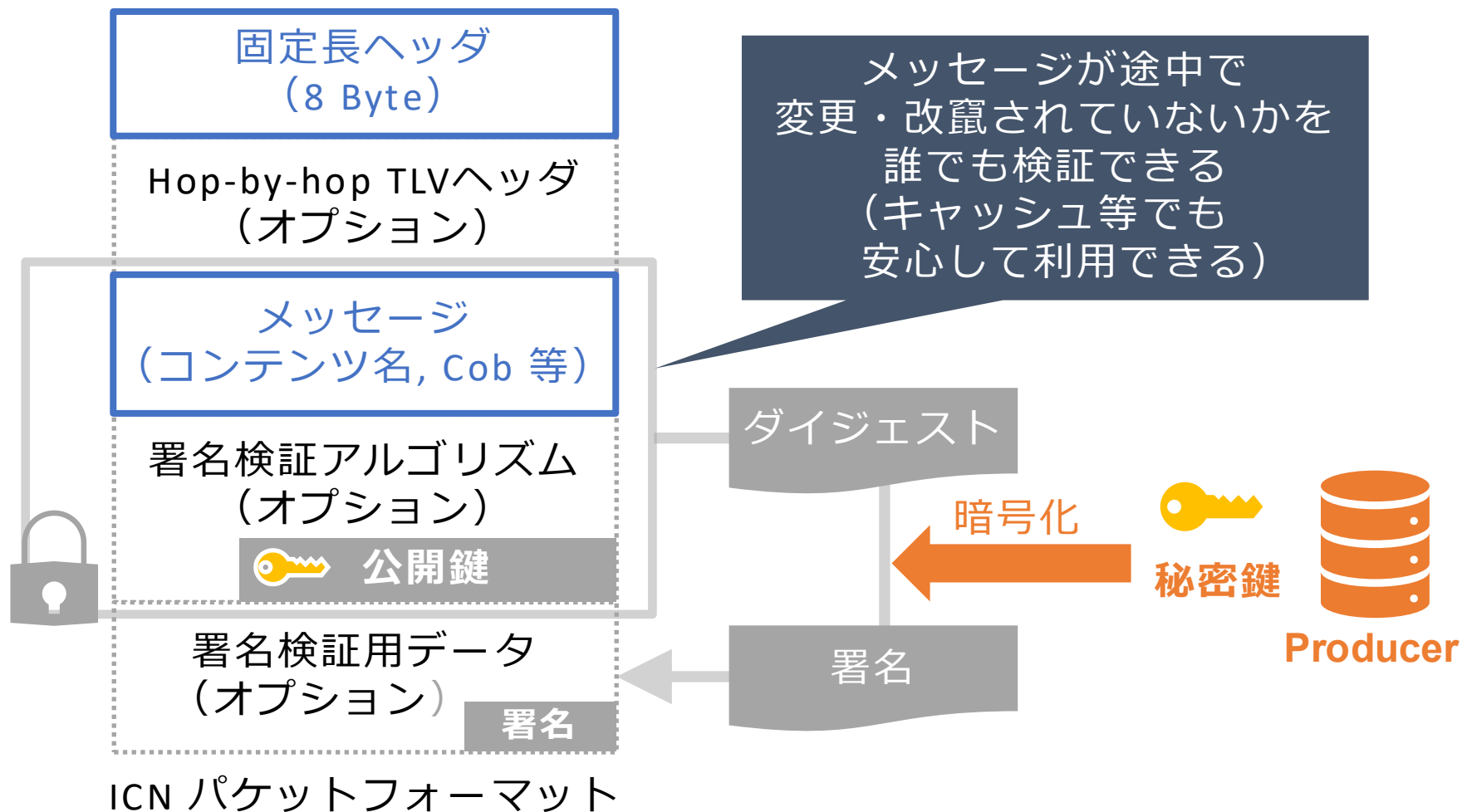
- Face : 外部と接続する論理的なインターフェース
- 制御部 : 名前検索を行い FIB・PIT・CS を参照・更新



ICN/CCN ルータ

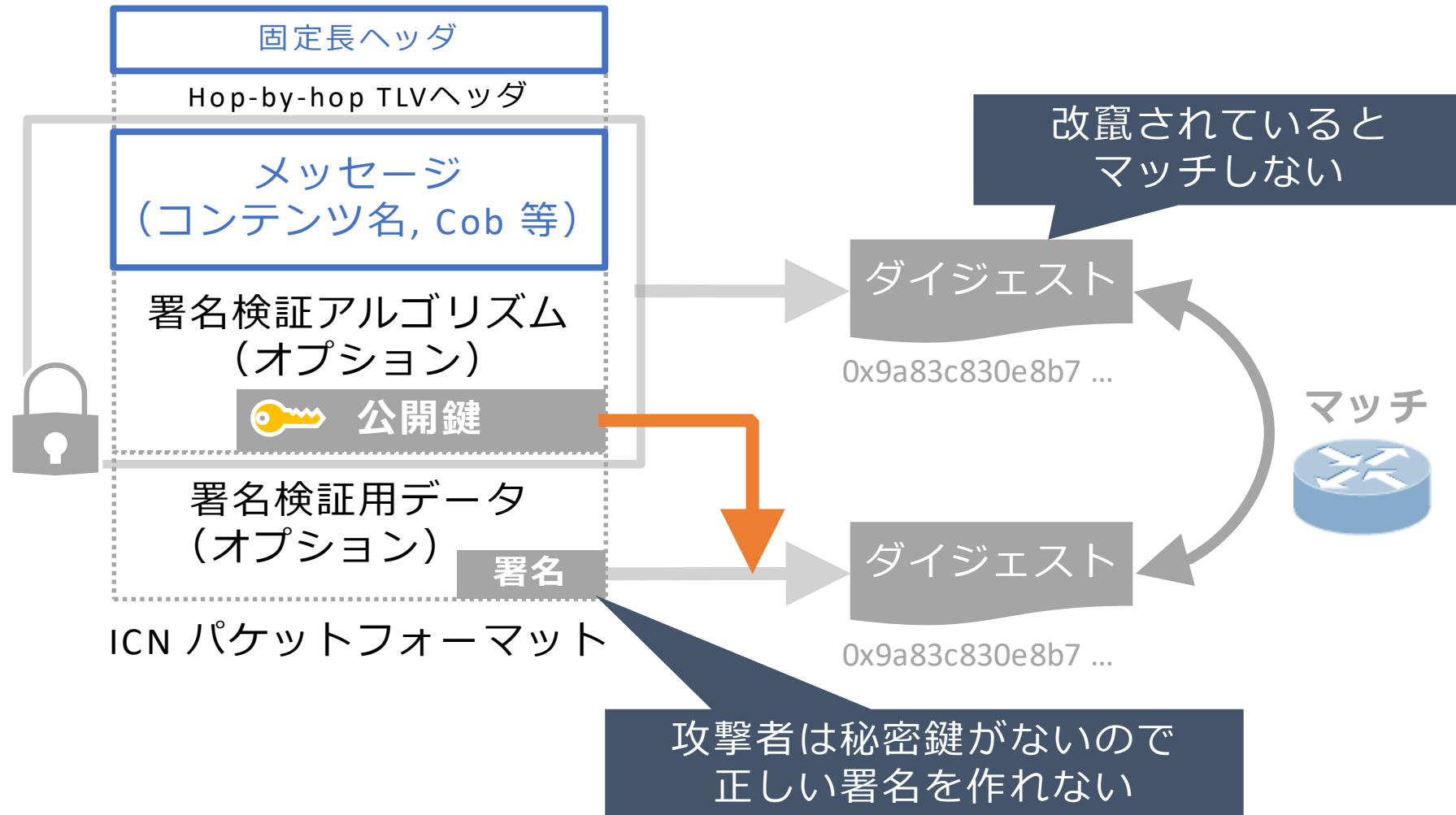
Data パケット単位のセキュリティ

■ デフォルトで自己署名機能を提供



自己署名の検証

- パケット内の公開鍵を使ってダイジェストを比較



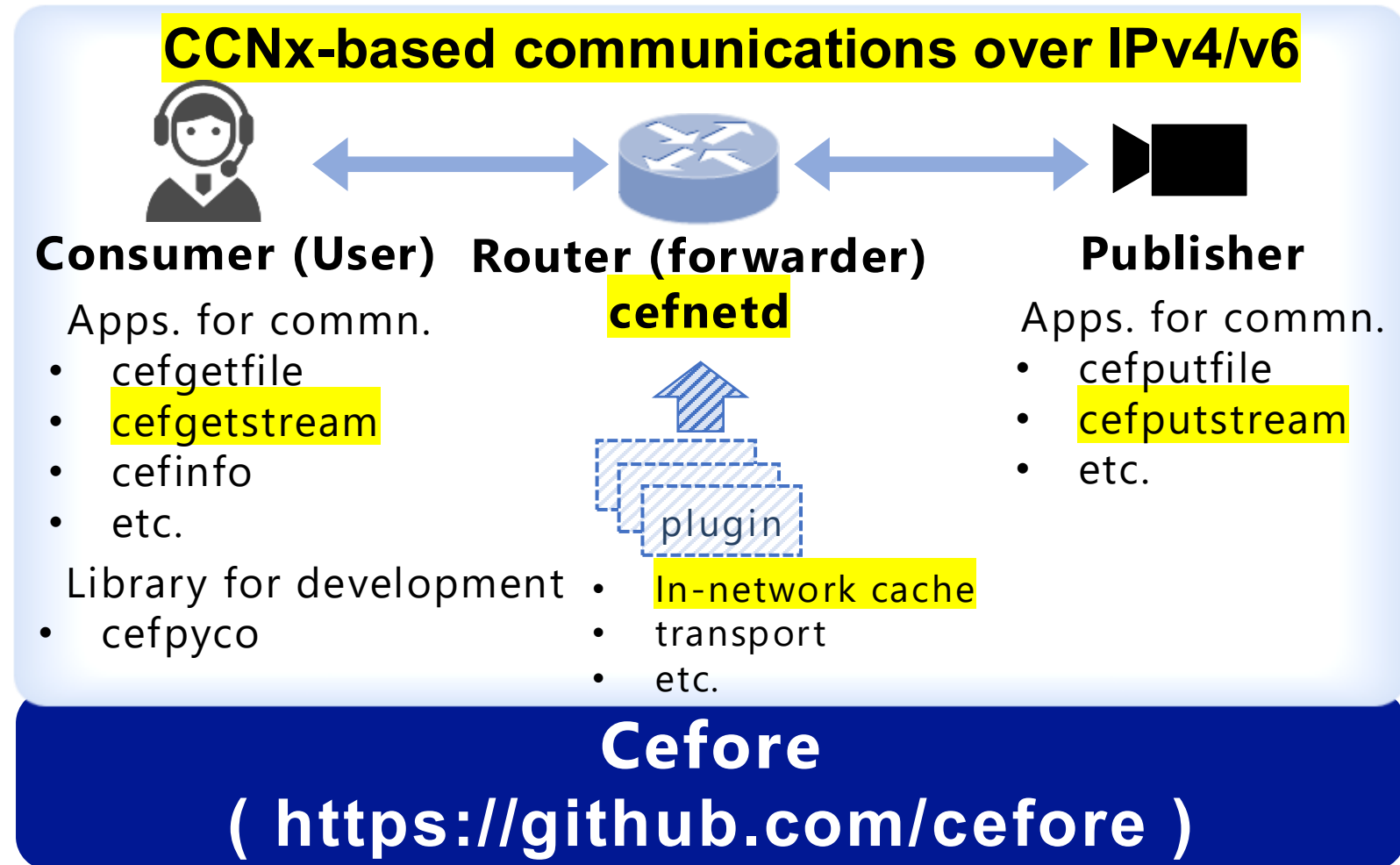
ICN が適した主要なユースケース

- コンテンツ配信・動画配信
 - キャッシュ活用
- IoT・IIoT (Industrial IoT)
 - 多対多通信・非同期データ共有・耐障害性
- エッジ/ネットワーク内コンピューティング
 - データ取得・キャッシュ・エッジ計算結果配信を統合
- モビリティ・車車間通信
 - 車両移動・ハンドオーバ・一時切断に対応
- 災害ネットワーク
 - 中央サーバ依存度を軽減、キャッシュ活用（断続的接続や動的トポロジへの適応）
- AI・分散学習基盤
 - マルチキャスト、キャッシュ活用、データ中心通信

Cefore

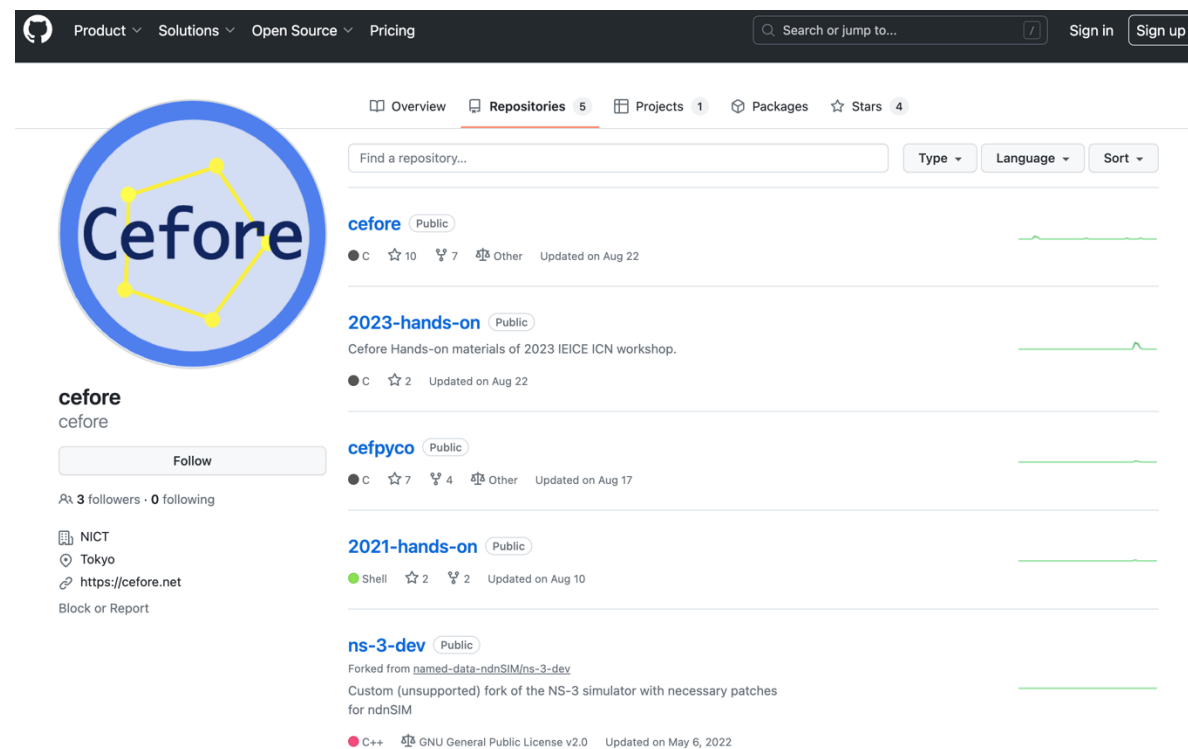
■ Cefore: ICN通信を行えるオープンソースプラットフォーム

- Compatible with CCNx-1.0 [RFC 8569] [RFC 8609]
- C language
 - Cefpyco library for app. development uses Python
- Supported OS
 - Linux (ubuntu 22.04 or later)
 - macOS (12 or later)
 - Raspberry Pi OS



GitLab でダウンロード可能

- Ceforeの解説（インストールの仕方など）や、チュートリアル資料(日本語)があります。
- 「cefore git」でググればすぐに出てきます。



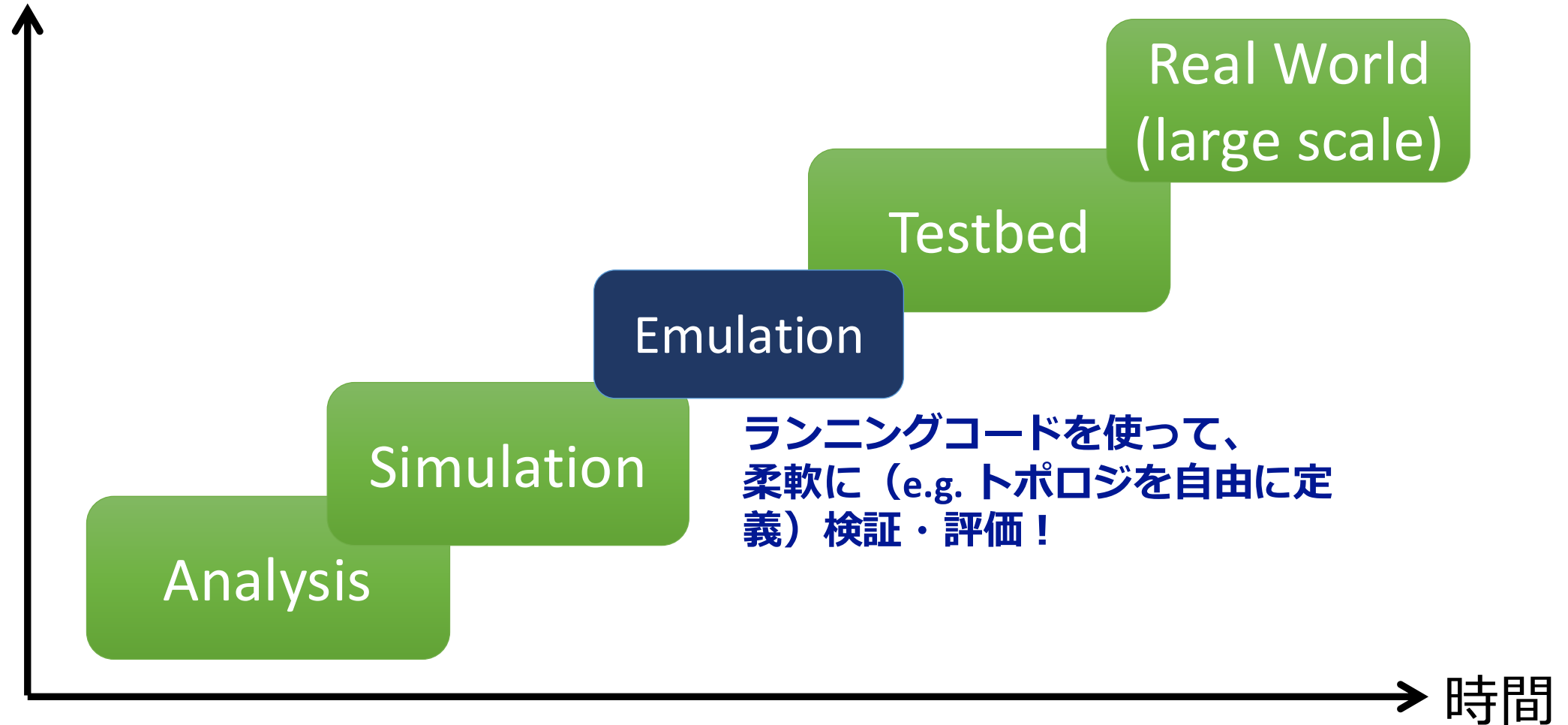
<https://github.com/cefore>

Cefore Emulator (CeforeEmu)

概要

評価ステップ

実装・バグ解消コスト



Mininetを利用

■ Mininet: (<https://mininet.org/>)

➤ 軽量

- 仮想マシンではなく Linux の namespace や cgroups を利用するため、**多数のノードを1台で動作可能。**

(CeforeEmu: 各ノード上でCeforeデーモンを実行)

➤ 実ネットワークに近い挙動

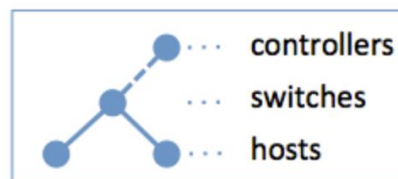
- 仮想ホスト上で実際のアプリケーションやプロトコルを実行可能。

Mininet

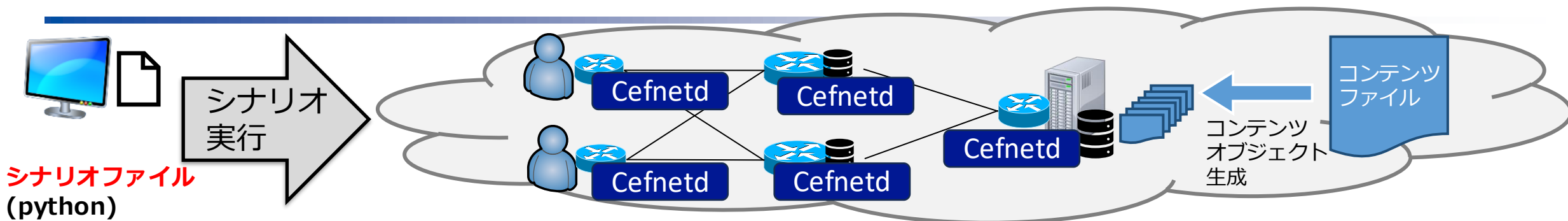
An Instant Virtual Network on your Laptop (or other PC)

Mininet creates a **realistic virtual network**, running **real kernel, switch and application code**, on a single machine (VM, cloud or native), in seconds, with a single command:

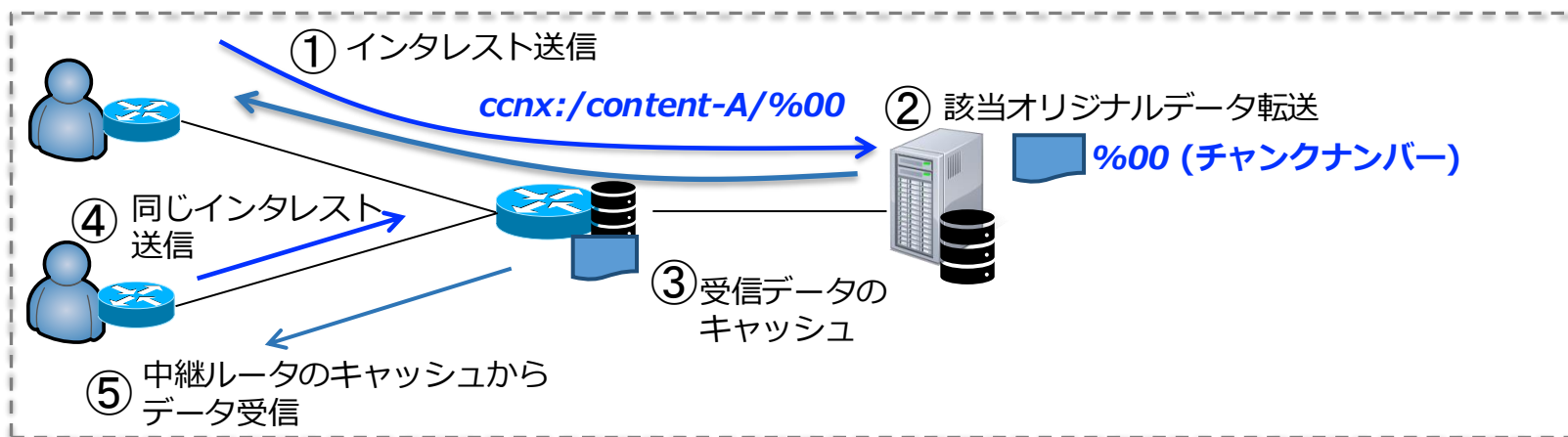
```
> sudo mn
```



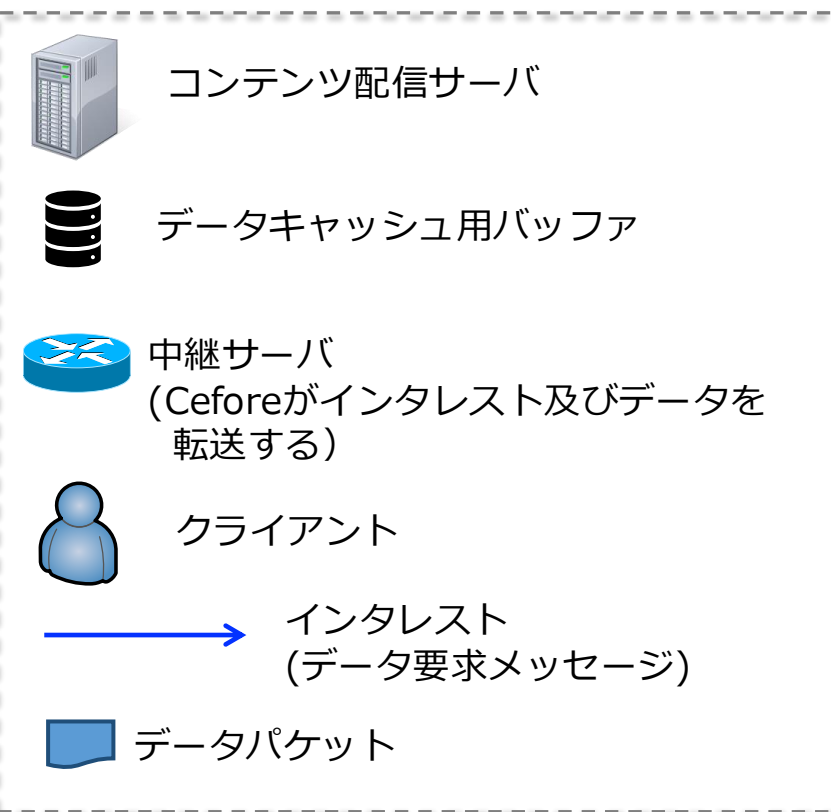
CeforeEmuのイメージと動作例



- ・ トポロジを自由に定義可能
- ・ どのノードに対して、いつ何を実行させるか定義可能 (Ceforeのみならず、他のアプリも勿論OK)



CeforeEmuを用いた“基本的”な動作例



3 ノードで実行してみよう

■ 1st ステップ

- Ceforeをダウンロードし、Ubuntu(24)にインストール
- <https://github.com/cefore> を参照

■ 2nd ステップ

- MininetをUbuntu(24)にインストール
 - <https://mininet.org/download/> の “Option 3: Installation from Packages” を参照
 - `$ mininet/util/install.sh -fw` ← **openflowの箇所でエラーが発生する。**
 - openflow/libの “util.h” と “util.c” を下記のように変更
 - “void strcpy()” → “size_t strcpy()”
 - openflowのディレクトリ直下で、`$ make` を実行し、`$sudo make install` でインストールする。
- テストコマンド： `$ sudo python3 ./mininet/example/multiping.py`

■ 3rd ステップ

- CeforeEmuを使って、3ノード (h0, h1, h2) を定義し、Publisher(h2)がファイルをPutし、Consumer(h0)がh1経由でファイルを取得
- <https://github.com/kazuhisa-v2/ICNwrkshps2026/Simple-3nodes> を参照
 - 任意のディレクトリ(e.g. Test-3nodes)を用意し、その直下に用意されているファイルとディレクトリ(h0,h1,h2)を配置し、シナリオスクリプトを実行

CeforeEmu実例

1. Reflexive Forwardingを 動作させる

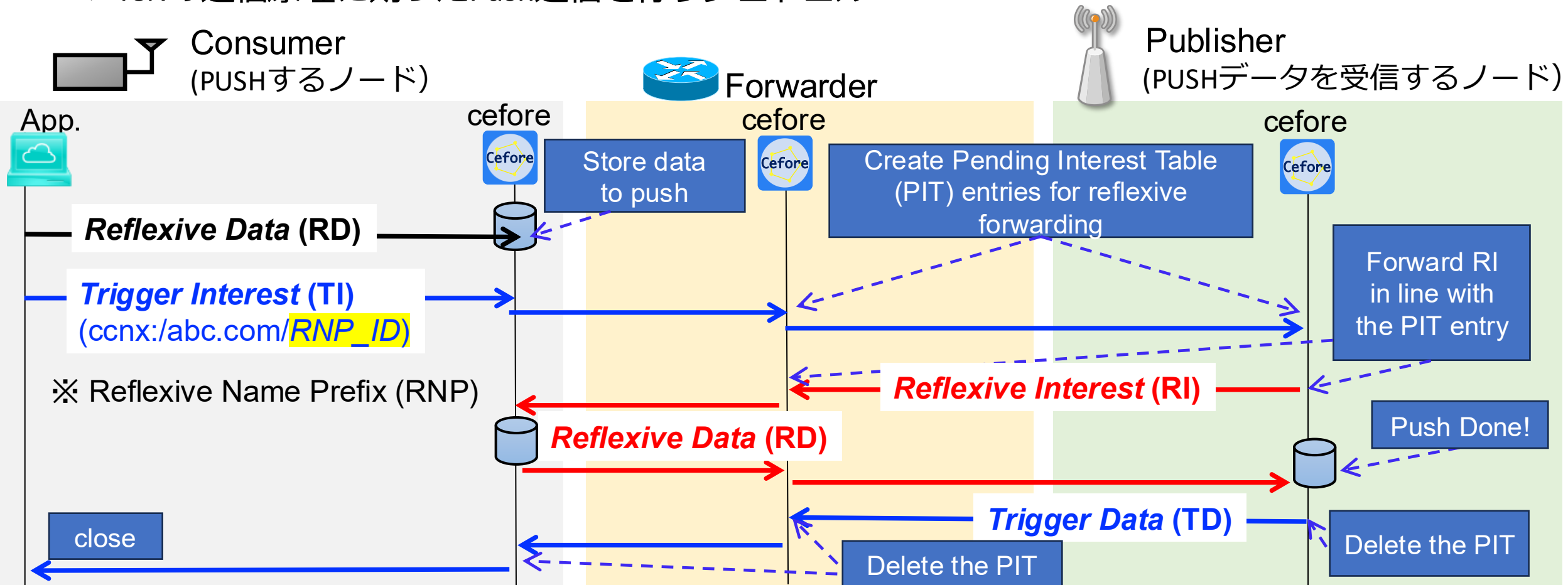
CeforeEmu: Reflexive Forwardingを実行

■ ICN通信

- 原則はPullベース通信（consumerがinterestを発行し、指定のデータを受信）
- Push通信も時には必要（e.g. センサーが任意のデータをゲートウェイにPush）

■ Reflexive Forwarding (<https://datatracker.ietf.org/doc/draft-irtf-icnrg-reflexive-forwarding/>)

- ICNの通信原理に則ったPush通信を行うプロトコル



CeforeEmu: Reflexive Forwardingを実行

■ 事前確認事項

- Consumerノードは、[/usr/local/cefore/cefnetd.conf](#)を編集し、**CS_MODE=1**にしてキャッシュを有効化しておく

■ まずは、1ノードで試してみよう

- Terminalを2つ開くと良いです (Terminal1 (Consumer)、Terminal2 (Publisher))
- 1) Publisher側でcefsbfileを実行 (ConsumerからのPush依頼を受け付ける用にする)
 - `$ cefsbfile ccnx:/push-data`
- 2) (cefsbfile待受中に) Consumer側でcefputfileを実行
 - `$ cefpubfile ccnx:/push-data -f ./Pushしたいデータのファイル名`
- 3) (PUSH完了時) cefsbfileのログ (Publisher側) に以下のような結果が出力される
 - [cefsbfile] Completed to get all the chunks.
 - [cefsbfile] RNP=0x2fc6eac653483f47eb5830131cadb24a
 - [cefsbfile] Outputfile: RNP0x2fc6eac653483f47eb5830131cadb24a.out
 - [cefsbfile] Rx Frames (All) = 2500
 - [cefsbfile] Rx Frames (ContentObject) = 2500
 - [cefsbfile] Rx Bytes (All) = 20619744
 - [cefsbfile] Rx Bytes (ContentObject) = 20480000
 - [cefsbfile] Duration = 0.537 sec
 - [cefsbfile] Throughput = 307895236 bps
 - [cefsbfile] Goodput = 305808571 bps
 - [cefsbfile] Jitter (Ave) = 214 us
 - [cefsbfile] Jitter (Max) = 84110 us
 - [cefsbfile] Jitter (Var) = 2997142 us

正常動作を確認したら、
CeforeEmuで実行してみよう

CeforeEmu実例

2. ワイヤレス環境で実行

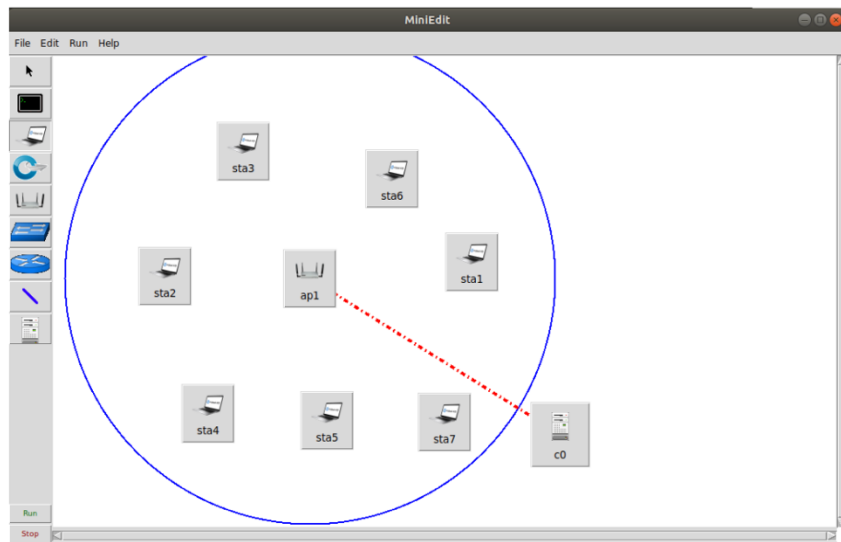
ワイヤレス環境を模擬

■ Mininet-WiFi (<https://mininet-wifi.github.io/get-started/>)

- Mininet を拡張し、無線LAN (Wi-Fi) 環境をエミュレーションできるオープンソースプラットフォーム
- 主要な特徴
 - Wi-Fiネットワークのエミュレーション
 - モビリティ（移動）の再現
 - 無線伝搬モデルの利用



■ Mininet-WiFiをインストールして試してみよう



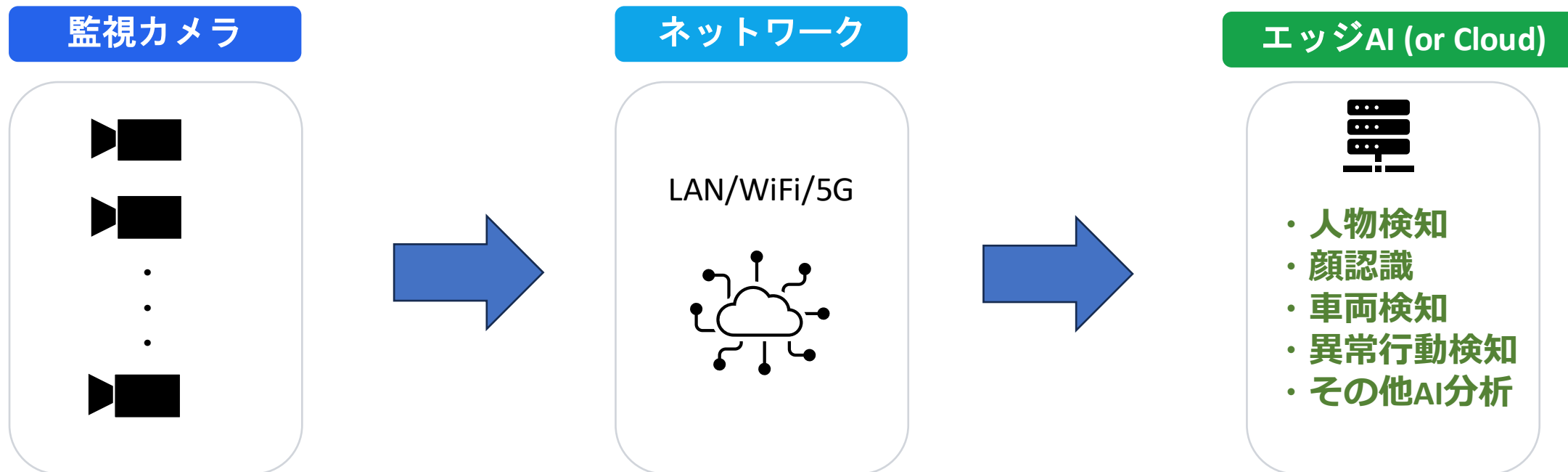
センサーネットワークをCeforeEmuで模擬して、ICN通信や、Reflexive Forwardingを使ったPUSH通信を実行してみよう

CeforeEmu実例

3. Vision AI/Computer Vision システムを動作させる

Vision-AI/Computer-vision System

- Vision-AIを「対象システム」として**ネットワーク側（帯域、レイテンシ、エッジクラウド分散、リソース割当）を最適化**する研究は、NSDI・SIGCOMM・INFOCOM・MobiCom/MobiSys といったトップ会議や、IEEE/ACM Transactions on Networking、IEEE Transactions on Mobile Computing、IEEE Communications Surveys & Tutorials といった著名論文誌で、**この2～3年でも継続的に取り扱われている活発な研究テーマ。**



手順

■ Step1: 事前準備

- 物体検知・追跡を行う環境を整える（Video Object Detection with YOLv8 + KITTI Dataset）

■ Step2: localテスト 1

- cefnetdを用いずに、映像配信・再生を実行

■ Step3: localテスト 2

- cefnetd 1 つを起動し、cefnetdを経由して、映像配信・再生を実行

■ Step4: localテスト 3

- cefnetd 1 つを起動し、cefnetdを経由して、映像配信・物体検知&追跡・処理結果再生を実行

■ Step5: CeforeEmuを使う

- Cefnetd3つ（h0:consumer, h1:router, h2:publisher）を用いて、映像配信、物体検知&追跡・処理結果再生を実行

Step1 事前準備:

Video Object Detection with YOLv8 + KITTI Dataset

■ ffmpegをインストール

- `$ sudo apt update → $ sudo apt install ffmpeg → $ ffmpeg -version`

■ Pythonの実行環境（仮想化環境）作成

- `$ sudo apt install python3-venv python3-pip`
- `$ mkdir ~/video-detection && cd ~/video-detection`
- `$ python3 -m venv venv → $ source venv/bin/activate`（仮想化環境の有効化）
 - 仮想化環境を終了するときは、`$ deactivate`
- `$ pip install --upgrade pip`

■ PyTorchのインストール

- `$ pip install torch torchvision`

■ Ultralytics (YOLOv8)のインストール

- `$ pip install ultralytics`
- `$ python3 -c "import ultralytics; ultralytics.checks()"`（versionが表示されればOK）

Step1 事前準備:

Video Object Detection with YOLOv8 + KITTI Dataset (Con't)

■ ビデオデータの準備

- `$ wget https://s3.eu-central-1.amazonaws.com/avg-kitti/raw_data/2011_09_26_drive_0005/2011_09_26_drive_0005_sync.zip`
- `$ wget https://s3.eu-central-1.amazonaws.com/avg-kitti/raw_data/2011_09_26_calib.zip`
- `$ unzip 2011_09_26_drive_0005_sync.zip && unzip 2011_09_26_calib.zip`

■ 画像シーケンスを動画に変換

- `$ ffmpeg -framerate 10 -pattern_type glob -i '2011_09_26/2011_09_26_drive_0005_sync/image_02/data/*.png' -vf "scale=1242:376" -c:v libx264 -pix_fmt yuv420p kitti_city.mp4`
 - KITTIの画像サイズは1242×375。H.264では画像の縦横サイズが偶数である必要があるため、高さを376ピクセルに拡大。

■ YOLOv8で物体検知を実行

Step1 事前準備:

Video Object Detection with YOLOv8 + KITTI Dataset (Con't)

■ YOLOv8で物体検知を実行

➤ 検知のみ (Per-Frame, No Tracking)

- ▣ `$ yolo detect predict model=yolov8m.pt source=kitti_city.mp4 save=True conf=0.3 save_txt=True`

- ▣ `$ ffplay runs/detect/predict/kitti_city.avi` (結果を動画で見る)

➤ 検知+追跡 (検出された各オブジェクトには、フレームをまたいで一貫したIDが割り当てられる)

- ▣ `$ yolo track model=yolov8m.pt source=kitti_city.mp4 save=True conf=0.3 tracker=bytetrack.yaml save_txt=True save_conf=True`

- ▣ `$ ffplay runs/detect/track/kitti_city.avi` (結果を動画で見る)

Step1 事前準備:

Video Object Detection with YOLv8 + KITTI Dataset (Con't)

■ 参考：他のKITTI画像シーケンスデータ

➤ Residential driving

- ❑ `$ wget https://s3.eu-central-1.amazonaws.com/avg-kitti/raw_data/2011_09_26_drive_0019/2011_09_26_drive_0019_sync.zip`
- ❑ `$ unzip 2011_09_26_drive_0019_sync.zip`
- ❑ `$ ffmpeg -framerate 10 -pattern_type glob -i '2011_09_26/2011_09_26_drive_0019_sync/image_02/data/*.png' -vf "scale=1242:376" -c:v libx264 -pix_fmt yuv420p kitti_residential.mp4`

➤ Longer city drive

- ❑ `$ wget https://s3.eu-central-1.amazonaws.com/avg-kitti/raw_data/2011_09_26_drive_0014/2011_09_26_drive_0014_sync.zip`
- ❑ `$ unzip 2011_09_26_drive_0014_sync.zip`
- ❑ `$ ffmpeg -framerate 10 -pattern_type glob -i '2011_09_26/2011_09_26_drive_0014_sync/image_02/data/*.png' -vf "scale=1242:376" -c:v libx264 -pix_fmt yuv420p kitti_city2.mp4`

➤ Run detection + tracking on both

- ❑ `$ yolo track model=yolov8m.pt source=kitti_residential.mp4 save=True conf=0.3 tracker=bytetrack.yaml`
- ❑ `$ yolo track model=yolov8m.pt source=kitti_city2.mp4 save=True conf=0.3 tracker=bytetrack.yaml`

Step2: localテスト 1

- cefnetdを用いずに、映像配信・再生を実行
 - kitti_city.mp4の映像データを、ffplayで受信・再生。
 - 1) `$ ffplay -fflags nobuffer -flags low_delay -probesize 32 -analyzeduration 0 udp://0.0.0.0:5000`
(受信ノードとしてのコマンド)
 - 2) `ffmpeg -re -i kitti_city.mp4 -c:v libx264 -preset ultrafast -g 30 -keyint_min 30 -f mpegts udp://localhost:5000` (送信ノードとしてのコマンド)
 - 動画再生のみなら、下記でOK
 - `$ ffplay kitti_city.mp4`
 - (※ kitti_city.mp4は動画時間が短いので、“Longer city drive”をdownloadして動画を準備すると良いかも)

Step3: localテスト 2

- cefnetd 1 つを起動し、そのcefnetdを経由して、映像配信・再生を実行
 - 1) \$cefnetdstart
 - \$ cefstatus を実行し、cefnetdが正常に起動しているか確かめよう
 - 2) cefgetstream ccnx:/live/video | ffplay -fflags nobuffer -flags low_delay -probesize 32 -analyzeduration 0 - (受信ノードとしてのコマンド)
 - 3) ffmpeg -re -i kitti_city2.mp4 -c:v libx264 -preset ultrafast -tune zerolatency -g 15 -bf 0 -an -f mpegts - | cefputstream ccnx:/live/video (送信ノードとしてのコマンド)
 - 4) \$ cefnetdstop
- cefnetdがうまく起動しないときは、[/tmp/cef_9695.0](#) が残っていないかチェック。残っていたら削除。

Step4: localテスト 3

- cefnetd 1 つを起動し、cefnetdを経由して、映像配信・物体検知&追跡・処理結果再生を実行
 - 流れ
 - 受信ノード : cefgetstream → ffmpeg (Decode) → Python → YOLOv8 + ByteTrack → OpenCV表示
 - 送信ノード : ffmpeg → cefputstream
 - 実行手順 :
 - \$ cefnetdstart
 - \$ python3 track_cefore.py (受信ノードとして実行。track_cefore.pyはgitからdownloadできます)
 - \$ ffmpeg -re -i kitti_city2.mp4 -c:v libx264 -preset ultrafast -tune zerolatency -g 5 -bf 0 -an -f mpegts - | cefputstream ccnx:/live/video
(上手くいけば、bounding boxとtrack-idが付与された映像が再生される)
 - \$ cefnetdstop
- 映像は再生せず、受信した映像データをファイルに保存し、YOLOで解析をする場合
 - 実行手順 :
 - \$ cefnetdstart
 - \$ cefgetstream ccnx:/live/stream > recv.ts
 - \$ yolo track model=yolov8m.pt source=recv.ts save=True conf=0.3 tracker=bytetrack.yaml save_txt=True save_conf=True
 - \$ cefnetdstop

Step 5: CeforeEmuを使う

- Cefnetd3つ（h0:consumer, h1:router, h2:publisher）を用いて、映像配信、受信映像保存、保存した映像に対して物体検知を実行し解析してみよう
- 受信映像をリアルタイムに解析できるようにしてみよう。